

Programación en lenguaje C

Proyecto: laberinto

La presente documentación se da en carácter explicativo del software adjunto, cuyo fin es dar una muestra de conocimientos adquiridos en el manejo del lenguaje C de programación. El mismo fue desarrollado en el entorno Linux Mint utilizando como herramienta Visual Studio Code.

El programa es un juego interactivo de tipo laberinto, diseñado para ejecutarse en la terminal de comandos en tiempo real, implementando un control de entrada directo y sin espera de retorno. El objetivo del jugador es moverse desde una posición inicial hasta hallar la salida en el menor número de movimientos posible, evitando paredes y utilizando las teclas W, A, S, D para moverse en las direcciones norte, oeste, sur y este respectivamente. El juego presenta una pantalla inicial a modo de menú con opciones y el jugador puede retornar al menú en cualquier momento presionando la tecla 'Q' y salir definitivamente eligiendo la opción designada desde el mismo. El programa está estructurado en varias funciones que cumplen roles específicos:

1.Configuración de Terminal: La función 'configurarTerminal()' y 'obtenerTecla()' se encargan de cambiar el modo de entrada de terminal para permitir la lectura de teclas individuales sin necesidad de presionar Enter. Esto se logra mediante la modificación de los atributos de terminal ('termios') para deshabilitar el modo canónico (donde se espera una línea completa) y el eco (donde se muestra lo que se escribe). La función 'obtenerTecla()' lee un solo carácter de entrada sin mostrarlo en pantalla, lo cual es esencial para brindar la experiencia de un juego en tiempo real.

2.Carga del Laberinto: La función 'cargarLaberinto()' lee un archivo de texto (por defecto "laberinto.txt") y lo carga en una matriz de caracteres de 17x33. Verifica que el archivo exista y que tenga exactamente 17 líneas, cada una con 33 caracteres (o menos, pero con un relleno adecuado). Si el archivo no se puede abrir o tiene un formato incorrecto, el programa muestra un mensaje de error y regresa al menú principal.

3.Cálculo de Movimientos: La función 'calcularMinimosMovimientos()' implementa un algoritmo de búsqueda en anchura (BFS) para encontrar el camino más corto desde la posición inicial (1,1) hasta la salida ('S'). Utiliza una cola de nodos (estructura 'Nodo' que contiene coordenadas 'x','y' y la distancia recorrida) para explorar el laberinto en capas. Inicia desde (1,1) y explora todos los vecinos válidos (no paredes, dentro de los límites del laberinto) en cada paso. Cuando encuentra la salida, devuelve la distancia mínima. Si no hay camino, retorna -1.

4.Interacción del Jugador: El jugador se mueve mediante las teclas W, A, S, D, y el estado del juego se actualiza en tiempo real en la pantalla. La función 'mostrarLaberinto()' dibuja el laberinto en la consola, marcando la posición actual del jugador ('J'). La función 'validarMovimiento()' verifica que la entrada sea una de las teclas permitidas. La función 'moverJugador()' actualiza las coordenadas del jugador según la tecla presionada, siempre que no haya una pared ('#') en la dirección de movimiento.

5.Flujo de Ejecución: El programa comienza en 'main()', donde se configura la terminal y se entra en un bucle infinito que muestra el menú de inicio. El usuario elige entre jugar o salir. Si elige jugar, el laberinto se carga, se calcula el camino mínimo presentando su cantidad de pasos como desafío al usuario, y se inicia el bucle secundario o bucle del juego. Durante el juego, el jugador se mueve hasta que llega a la salida ('S') o decide salir presionando 'Q'. Si tiene éxito, se presenta un mensaje de felicitación junto a la cantidad de movimientos realizados. Al salir del juego, el programa regresa al menú principal. El bucle principal se mantiene activo hasta que el usuario elige salir completamente del programa desde el menú.

6.Limpieza de Recursos: Al final del programa, la función 'main()' restaura la configuración original del terminal para asegurar que el usuario pueda utilizar la consola de manera tradicional después de que el programa termine.

El programa combina elementos de programación de sistemas (control de terminal), algoritmos de búsqueda (BFS), y programación de juegos (interacción en tiempo real) y se propone como un ejemplo de cómo se pueden integrar múltiples conceptos de programación en un solo proyecto funcional.